DEPARTMENT OF MATHEMATICS & COMPUTER SCIENCE

Faculty of Science and Technology · National University of Lesotho

CS3520

Computer Organisation and Architecture I

Week 3 — The RISC-V Instruction Set Architecture I: Operands, Instructions & Encoding

Mr K. O. Setetemela

Course Coordinator · Department of Mathematics & Computer Science

Academic Year 2026/2027 · Semester A

RECAP & THIS WEEK

From Measuring to Instructing

Weeks 1–2: we saw that programs become machine instructions, and learned to measure how long those instructions take. Now we open the instruction set itself.

1	**ISA Design Principles**	Why RISC-V looks the way it does
2	**Registers & Operands**	The 32 registers and their conventions
3	**Arithmetic & Logical Instructions**	add, sub, addi, and the shifts
4	**Memory Operands**	lw, sw, byte addressing and alignment
5	**Instruction Formats**	R-type, I-type, S-type
6	**Encoding by Hand**	Assembly ⟶ 32 bits, field by field

WHY THIS ISA LOOKS LIKE THIS

Three Design Principles

Every choice in the RISC-V instruction set is a deliberate engineering trade-off. Three principles explain almost all of them.



Simplicity Favours Regularity

Every instruction is exactly 32 bits. Arithmetic always takes exactly three operands. Regular hardware is simpler — and simpler hardware is faster.



Smaller Is Faster

Only 32 registers, not 1000. A small register file can be read quickly; a large one cannot. Scarcity is the price of speed.



Good Design Demands Good Compromises

A fixed 32-bit instruction cannot hold every field we might want. RISC-V keeps register fields in fixed positions and splits the immediate instead.

WHERE THE WORK HAPPENS

The 32 Registers

Arithmetic operates on registers, never directly on memory. RISC-V gives you exactly 32, each 32 bits wide — and by convention, each has a job.

Register	ABI Name	Conventional Use
x0	zero	Hard-wired to 0 — always. Writes are discarded.
x1	ra	Return address — set automatically by jal
x2	sp	Stack pointer
x5–x7, x28–x31	t0–t6	Temporaries — a called procedure may destroy these
x8–x9, x18–x27	s0–s11	Saved — a called procedure must preserve these
x10–x17	a0–a7	Arguments and return values

These names are a convention, not hardware — but break them and your code will not interoperate with anyone else's.

THE WORKING INSTRUCTIONS

Arithmetic & Logical Operations

Every arithmetic instruction has the same shape: one destination, two sources. Simplicity favours regularity.

```
# C:  f = (g + h) - (i + j);

add  t0, s1, s2     # t0 = g + h
add  t1, s3, s4     # t1 = i + j
sub  s0, t0, t1     # f  = t0 - t1
```

The “i” suffix means immediate — a constant baked into the instruction itself, rather than read from a register. addi t0, t0, 4 adds four without touching memory.

Arithmetic

`add` `sub` `addi`

Logical

`and` `or` `xor` `andi` `ori` `xori`

Shifts

`sll` `srl` `sra` `slli` `srli` `srai`

Comparison

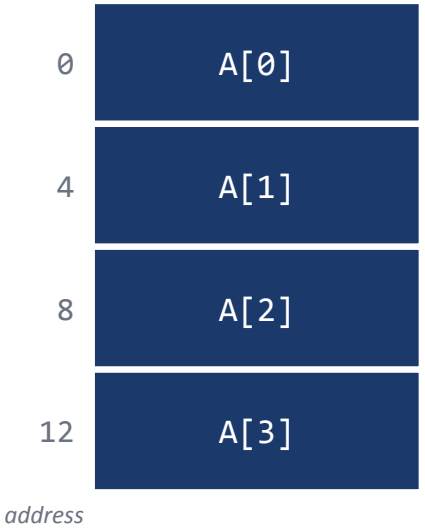
`slt` `sltu` `slti`

REACHING BEYOND THE REGISTERS

Memory Operands: Load & Store

Only two instructions touch memory. Everything else works in registers — this is what makes RISC-V a load–store architecture.

MEMORY (byte addressed)



Words are 4 bytes apart — so A[2] lives at offset 8, not 2.

```
# C:  A[2] = A[0] + 12;

lw   t0, 0(s1)   # load  A[0]
addi t0, t0, 12
sw   t0, 8(s1)   # store A[2]
```

Base + offset

The only addressing mode for data: a register plus a constant.

Alignment

Word addresses should be multiples of 4.

s1 holds the base

The array's starting address lives in a register.

INSIDE THE 32 BITS

Instruction Format: R-Type

Register-to-register arithmetic. Three register fields, plus two function fields that pin down exactly which operation is meant.



`opcode`	Which family of instruction — and therefore which format to decode.
`rd`	Destination register — where the result is written.
`rs1, rs2`	The two source registers.
`funct3 / funct7`	Narrow the opcode down to one exact operation. add and sub share an opcode and funct3 — only funct7 tells them apart.

The register fields never move between formats — the decoder can start reading registers before it has finished identifying the instruction.

INSIDE THE 32 BITS

Instruction Formats: I-Type & S-Type

I-TYPE — immediates and loads (addi, andi, lw, jalr)



One source register is replaced by a 12-bit constant — sign-extended to 32 bits before use.

S-TYPE — stores (sw, sh, sb)



Why is the S-type immediate split in two? A store has no destination register — but rather than shift rs1 and rs2 to fill the gap, RISC-V keeps them exactly where R-type and I-type put them, and splits the immediate around them instead. Good design demands good compromises: harder for the assembler, faster for the hardware.

WORKED EXAMPLE

Encoding an Instruction by Hand

Translate `add x7, x5, x6` into the 32 bits the processor actually executes.

1. **Identify the format** — `add is register-to-register → R-type`
2. **Look up the fields** — `opcode = 0110011,  funct3 = 000,  funct7 = 0000000`
3. **Convert the registers** — `rd = x7 = 00111,  rs1 = x5 = 00101,  rs2 = x6 = 00110`
4. **Assemble the fields in order**




```
0000 0000 0110 0010 1000 0011 1011 0011   =   0x006283B3
```

THINK ABOUT IT

RISC-V has no instruction that adds two values in memory directly — you must load, add, then store.

That is three instructions where some older architectures needed one. So why would a modern ISA deliberately make the programmer do more work? What does the hardware gain in exchange?

Discuss with a neighbour for two minutes before we continue.

HANDS-ON THIS WEEK

Lab 2 — Your First Assembly Programs

Translate by hand

Take three short C statements and write the equivalent RISC-V assembly.

Assemble and trace

Run them in RARS. Step instruction by instruction and watch the registers change.

Encode and verify

Hand-encode one instruction, then check your answer against the machine code column.

BEFORE YOU ARRIVE

- Pull the latest course repository
- Have RARS open and working
- Bring the register-convention table — you will need it

Tutorial Set 2 is issued this week: format identification and hand-encoding practice. Not collected — use any resource, including AI tools.

CHECKPOINT · CLO2

Key Takeaways

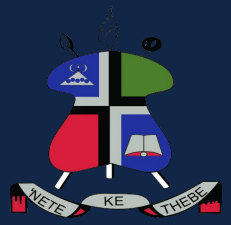
1. Explain the three design principles behind RISC-V, and point to where each one shows up in the instruction set.

2. Use the 32 registers correctly, following the ABI naming conventions.

3. Write arithmetic, logical and load–store sequences that implement a given C statement.

4. Identify R-type, I-type and S-type instructions, and encode one by hand into 32 bits.

Essential reading: Patterson & Hennessy, Ch. 2 (§2.1–§2.5) • Stallings, Ch. 12–13 • RISC-V card in the course repository



QUESTIONS?

COMING UP — WEEK 4

The RISC-V Instruction Set Architecture II

Control flow, procedures, the stack & calling conventions

Mr K. O. Setetemela • ko.setetemela@nul.ls